

Data Mining practice 11

Task1. Train.csv 파일을 trainset & validation set 둘로 임의로 나눈 다음 logistic regression 기반의 multiclass 분류기를 만들고 validation set을 사용하여 성능을 측정하여보라

```
phone <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/12week practice/train.csv")
head(phone)
summary(phone)
```

우선은 데이터를 다운받았다. train데이터이지만 총 7352개 행, 563개 열이 있다. 이중에서 우리는 activity열의 값을 예측을 해야 한다.

```
# Target 변수 변환 (factor 형태로 변환)
phone$Activity <- as.factor(phone$Activity)
```

우선 예측하고자 하는 열의 값들을 factor형태로 변환한다.

```
# Train/Validation Split (80:20 비율)
set.seed(2024) # 결과 재현성을 위해 시드 설정
trainIndex <- createDataPartition(phone$Activity, p = 0.8,
list = FALSE)
trainData <- phone[trainIndex, ]
validationData <- phone[-trainIndex, ]
```

다음 train과 test를 분리한다. 시드는 20224로 정하고, 0.8의 비율로 분리하였다.

```
# Logistic Regression 모델 생성 (모든 변수 사용)
logistic_model <- multinom(Activity ~ ., data = trainData)
```

다음은 모든 변수를 사용하여 로지스틱 회귀모델을 생성하려고 하였으나,

```
> logistic_model <- multinom(Activity ~ ., data = trainData)
nnet.default(X, Y, w, mask = mask, size = 0, skip = TRUE, softmax = TRUE,
에서 다음과 같은 에러가 발생했습니다:
가중치의 개수 (3384)가 너무 많습니다.
```

변수가 너무 많아서 그런 것인지 모델 생성에 실패했다. 결국 중요도에 따라서 feature을 제거하는 방법을 통해 변수를 조금 줄여야 한다고 판단했다

```
#특성 중요도 확인
library(randomForest)
# Random Forest 모델 학습
rf_model <- randomForest(Activity ~ ., data = trainData, importance = TRUE)
```

특성의 중요도를 확인하기 위해 random forest모델을 학습시켜 중요도를 확인해보고자 하였다

```
# 특성 중요도 확인
importance <- importance(rf_model)
print(importance)
```

이제, 이 중요도가 높은 상위 10개의 feature만을 가지고 이전에 문제가 생겼던 학습을 계속 하고자 한다

```
# 특성별 평균 중요도 계산
importance_df <- as.data.frame(importance)
mean_importance <- rowMeans(importance_df)
```

```

# 중요도 높은 순으로 정렬
sorted_importance <- sort(mean_importance, decreasing = TRUE)

# 상위 10개 특성 선택
top_features <- names(sorted_importance[1:10])

# 상위 10개 특성만으로 모델 재학습
trainData_reduced <- trainData[, c(top_features, "Activity")]
validationData_reduced <- validationData[, c(top_features, "Activity")]

# Logistic Regression 재학습
logistic_model_reduced <- multinom(Activity ~ ., data = trainData_reduced, maxit = 1000)

```

```

> logistic_model_reduced <- multinom(Activity ~ ., data = trainData_reduced, maxit = 1000)
# weights: 72 (55 variable)
initial value 10542.712717
iter 10 value 5840.772641
iter 20 value 5094.277816
iter 30 value 4548.523293
iter 40 value 4468.960231
iter 50 value 4428.351139
iter 60 value 4393.049012
iter 70 value 4365.856584
iter 80 value 4346.059829
iter 90 value 4331.017455
iter 100 value 4317.028791
iter 110 value 4299.410960
iter 120 value 4295.537191
iter 130 value 4284.658155
iter 140 value 4270.233153
iter 150 value 4248.296892
iter 160 value 4219.274486

```

```
iter 170 value 4206.298073
iter 180 value 4190.690093
iter 190 value 4179.661244
iter 200 value 4173.891943
iter 210 value 4172.442806
iter 220 value 4170.969708
iter 230 value 4170.159388
iter 240 value 4169.669565
iter 250 value 4168.949793
iter 260 value 4168.257401
iter 270 value 4167.677712
iter 280 value 4166.674895
iter 290 value 4166.133810
iter 300 value 4165.654522
iter 310 value 4165.190086
iter 320 value 4165.102884
iter 330 value 4165.038045
final value 4164.938258
converged
```

iter 330부근에서 수렴했고, 최적화가 완료되었다. 그러나 initial value가 많이 떨어지기는 했으나 아직도 높은 것 같다

```
# Validation 예측 및 평가
predictions <- predict(logistic_model_reduced, validationData_
ta_reduced)
conf_matrix <- confusionMatrix(predictions, validationData_
reduced$Activity)
print(conf_matrix)
```

이제 validation에 대해 예측 후 평가를 확인하면

```
> print(conf_matrix)
Confusion Matrix and Statistics
```

Reference

Prediction		LAYING	SITTING	STANDING	WALKING	WALKING_DOWNSTAIRS	WALKING_UPSTAIRS
LAYING		280	0	0	0		
0	0						
SITTING		1	205	30	11		
9	8						
STANDING		0	46	212	69		
3	11						
WALKING		0	2	12	118		
38	25						
WALKING_DOWNSTAIRS		0	4	9	19		
125	33						
WALKING_UPSTAIRS		0	0	11	28		
22	137						

Overall Statistics

Accuracy : 0.7337

95% CI : (0.7102, 0.7561)

No Information Rate : 0.1914

P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.6789

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: LAYING	Class: SITTING	Class: STANDING	Class: WALKING
Sensitivity	0.9964	0.7977		
0.7737		0.48163		
Specificity	1.0000	0.9513		
0.8920		0.93704		
Pos Pred Value	1.0000	0.7765		
0.6217		0.60513		
Neg Pred Value	0.9992	0.9568		
0.9450		0.90024		

Prevalence	0.1914	0.1751
0.1866	0.16689	
Detection Rate	0.1907	0.1396
0.1444	0.08038	
Detection Prevalence	0.1907	0.1798
0.2323	0.13283	
Balanced Accuracy	0.9982	0.8745
0.8328	0.70934	

Class: WALKING_DOWNSTAIRS Class: WALKI

NG_UPSTAIRS

Sensitivity	0.63452
0.64019	
Specificity	0.94886
0.95136	
Pos Pred Value	0.65789
0.69192	
Neg Pred Value	0.94366
0.93937	
Prevalence	0.13420
0.14578	
Detection Rate	0.08515
0.09332	
Detection Prevalence	0.12943
0.13488	
Balanced Accuracy	0.79169
0.79577	

정확도는 73%정도임을 확인할 수 있었다.

이번에는 top_10개가 아닌, top_20개로 feature을 늘려보면 어떻게 되는지 확인하고자 한다

```
# 상위 10개 특성 선택
top_features <- names(sorted_importance[1:20])
```

```
# 상위 10개 특성만으로 모델 재학습
trainData_reduced <- trainData[, c(top_features, "Activity")]
validationData_reduced <- validationData[, c(top_features, "Activity")]

# Logistic Regression 재학습
logistic_model_reduced <- multinom(Activity ~ ., data = trainData_reduced, maxit = 1000)

# Validation 예측 및 평가
predictions <- predict(logistic_model_reduced, validationData_reduced)
conf_matrix <- confusionMatrix(predictions, validationData_reduced$Activity)
print(conf_matrix)
```

```
> print(conf_matrix)
Confusion Matrix and Statistics
```

Prediction		Reference				
		LAYING	SITTING	STANDING	WALKING	WALKING_DOWNSTAIRS
LAYING	281	0	0	0	0	
0	0	0	219	37	0	
SITTING	0	0	38	237	0	
0	0	0	0	0	230	
5	5	0	0	0	6	
WALKING_DOWNSTAIRS	186	8	0	0	9	
WALKING_UPSTAIRS	6	201	0	0	0	

```
Overall Statistics
```

Accuracy : 0.9223
 95% CI : (0.9075, 0.9355)
 No Information Rate : 0.1914
 P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.9065

McNemar's Test P-Value : NA

Statistics by Class:

	Class: LAYING	Class: SITTING	Class: STANDING	Class: WALKING
Sensitivity	1.0000	0.8521		
	0.8650	0.9388		
Specificity	1.0000	0.9694		
	0.9682	0.9918		
Pos Pred Value	1.0000	0.8555		
	0.8618	0.9583		
Neg Pred Value	1.0000	0.9686		
	0.9690	0.9878		
Prevalence	0.1914	0.1751		
	0.1866	0.1669		
Detection Rate	0.1914	0.1492		
	0.1614	0.1567		
Detection Prevalence	0.1914	0.1744		
	0.1873	0.1635		
Balanced Accuracy	1.0000	0.9108		
	0.9166	0.9653		

	Class: WALKING_DOWNSTAIRS	Class: WALKING_UPSTAIRS
Sensitivity		0.9442
		0.9393
Specificity		0.9890
		0.9880
Pos Pred Value		0.9300
		0.9306
Neg Pred Value		0.9913

0.9896	
Prevalence	0.1342
0.1458	
Detection Rate	0.1267
0.1369	
Detection Prevalence	0.1362
0.1471	
Balanced Accuracy	0.9666
0.9636	

정확도가 0.92%로 매우 높게 나온 것 같다.

이제 실제 test데이터에 대한 예측을 수행해보자

```
test<- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터
마이닝 실습/Data-Mining-Practicum/myR/12week practice/test.cs
v")
test$Activity

# 2. 테스트 데이터에서 중요 특성만 선택
test_reduced <- test[, c(top_features, "Activity")] # 중요
특성과 Activity 열 포함

# 3. Activity를 factor로 변환
test_reduced$Activity <- as.factor(test_reduced$Activity)

# 4. 예측 수행
test_predictions <- predict(logistic_model_reduced, test_re
duced[, top_features]) # Activity 열 제외

# 5. 성능 평가 (Confusion Matrix)
test_conf_matrix <- confusionMatrix(test_predictions, test_
reduced$Activity)
print(test_conf_matrix)

# 6. 정확도 출력
```

```
cat("Test Accuracy:", test_conf_matrix$overall["Accuracy"],
"\n")
```

Confusion Matrix and Statistics

Prediction		Reference				
		LAYING	SITTING	STANDING	WALKING	WALKING_DOWNSTAIRS
LAYING	537	0	0	0	0	
SITTING	0	371	72	0	0	
STANDING	0	119	460	0	0	
WALKING	0	0	0	468	0	
WALKING_DOWNSTAIRS	0	0	0	11	337	
WALKING_UPSTAIRS	0	1	0	17	410	

Overall Statistics

Accuracy : 0.8765
 95% CI : (0.8641, 0.8882)
 No Information Rate : 0.1822
 P-Value [Acc > NIR] : < 2.2e-16

Kappa : 0.8515

Mcnemar's Test P-Value : NA

Statistics by Class:

	Class: LAYING	Class: SITTING	Class: STANDING	Class: WALKING
Sensitivity	1.0000	0.7556	0.8647	0.9435

Specificity	1.0000	0.9707
0.9503	0.9698	
Pos Pred Value	1.0000	0.8375
0.7931	0.8635	
Neg Pred Value	1.0000	0.9521
0.9696	0.9884	
Prevalence	0.1822	0.1666
0.1805	0.1683	
Detection Rate	0.1822	0.1259
0.1561	0.1588	
Detection Prevalence	0.1822	0.1503
0.1968	0.1839	
Balanced Accuracy	1.0000	0.8631
0.9075	0.9567	

Class: WALKING_DOWNSTAIRS Class: WALKI

NG_UPSTAIRS	
Sensitivity	0.8024
0.8705	
Specificity	0.9885
0.9721	
Pos Pred Value	0.9208
0.8559	
Neg Pred Value	0.9678
0.9753	
Prevalence	0.1425
0.1598	
Detection Rate	0.1144
0.1391	
Detection Prevalence	0.1242
0.1625	
Balanced Accuracy	0.8955
0.9213	

```

> # 6. 정확도 출력
> cat("Test Accuracy:", test_conf_matrix$overall["Accurac
y"], "\n")
Test Accuracy: 0.8764846

```

테스트 정확도는 0.87로 높게 나온 것 같다.